

The Ingestion Gap

A ground-truth audit of AI Job Agent's job-sourcing pipeline: what the registry actually contains, what the current n8n workflows can actually ingest, and what architecture closes the distance between them.

Branch **feat/source-intelligence-integration** HEAD **4f5b232**

Live DB verified **2026-09-26**

Scope **investigation only — nothing modified**

Contents

01 Executive summary

02 Confirmed current state

03 Company source coverage

04 Current ingestion architecture

05 The main problem

06 Options considered

07 Recommended architecture

08 Why this architecture

09 What we should not do

10 LinkedIn / Apify decision

11 Source registry changes needed

12 n8n workflow audit

13 P0 blockers

14 P1 before production

15 P2 — later

16 Recommended next step

17 Implementation sequence

18 Business risk assessment

19 Questions for founder decision

20 Final verdict

01 Executive summary

2.9%

OF REGISTRY
INGESTIBLE BY
CODE THAT EXISTS
TODAY

593

COMPANY
SOURCES IN THE
LIVE REGISTRY

0

REAL JOBS EVER
INGESTED IN
PRODUCTION

The registry-building effort was not wasted, but it solved a different problem than ingestion. Of 593 live `company_sources` rows, only **17** use an ATS the codebase has a working adapter for (Greenhouse, Lever, Workable). A further 58 use a named ATS (Oracle, SAP, Workday, SmartRecruiters...) nobody has built an adapter for yet. The remaining **~500 sources (87%)** have no structured ATS at all — closing that gap requires a genuinely different mechanism, not more registry research.

Nothing ingests jobs in production today. All five n8n workflows in the repo are manual-trigger or inactive; the one ingestion pipeline that exists (Job Ingestion — Pilot Orchestrator) is well-engineered — real idempotency, real bounded retries, a real dedup-collision fix — but hardcoded to exactly four sources and has never run outside a local pilot. Nothing downstream of `jobs` exists either: matching, cover-letter generation, and application sending are all unbuilt.

Separately, and materially: the shared n8n instance already has three undocumented workflows created *today* — an AI web-scraping agent, a LinkedIn-via-Apify enrichment flow, and a Google Maps lead generator — none in version control, none reviewed. Someone has started answering this audit's central question outside the process meant to govern it.

Recommendation in one line: activate what's already built and tested against the real registry query (Section 16), then expand ATS-adapter coverage before spending any engineering effort on AI/browser/Apify for the long tail — and treat the LinkedIn/Apify question as a founder decision, not a fait accompli.

02 Confirmed current state

Repository

- Branch `feat/source-intelligence-integration`, HEAD `4f5b232`. In-flight, uncommitted work: a large behavioral rewrite of `source-intelligence-analyzer.ts`, three new migrations adding promotion/retry-eligibility logic to source intelligence, a new (untracked) Error Handler workflow, and matching test updates.
- `docs/job-ingestion-database-readiness-audit.md` (2026-09-14) is the most reliable prior-art document. `docs/job-source-discovery/discovery-report.md` is explicitly stale — its own successor audit says so — and should never be cited as current registry state again.

Database (live, queried directly)

- **559** companies, **593** company sources, **60** source-intelligence observations, **20** jobs — all 20 from one local pilot test run, zero from production.
- Zero rows anywhere reference *International Remote* as a target market, despite it being a named product market in `AGENTS.md`. The country breakdown sums exactly to 593 across only the five physical markets — the CSV research round that covered remote roles was never imported.

n8n — repo vs. live

- 5 workflows exist in the repo. **15 exist on the live, shared n8n instance**. The instance also runs two unrelated products (a "KnowledgeFlow AI" RAG assistant, a gold-price alert bot).
- **CV Analysis Worker has drifted**: the live, active workflow is named "CV Analysis Worker v2" with a different trigger-node name than the committed file. The repo is not an accurate record of what CV analysis is actually doing in production.
- **Source Intelligence Analyzer is active live** (daily schedule) while its source is mid-edit, uncommitted, locally.
- Three workflows were created *today*, untracked: an AI web-scraping agent, a LinkedIn-profile-enrichment-via-Apify flow, and a Google Maps lead generator — see Section 10.
- A Telegram credential named `ai_job_agent_alerts_bot` exists on the instance, contradicting the Error Handler's own code comment that no such credential exists.

What has zero implementation

Matching, cover-letter generation, and application sending have no code anywhere in the repo, despite the schema for `matches` / `cover_letters` / `applications` / `notifications` already existing since 2026-08-09. This is a bigger gap than ingestion coverage and is not solved by any ingestion architecture decision.

03 Company source coverage

Real counts, queried live, not estimated from CSV-era docs.

Review & automation status (n = 593)

REVIEW STATUS	COUNT	AUTOMATION ELIGIBILITY (SAME ROWS)
Verified	398	75 suitable_public_ats · 260 suitable_public_html · 43 manual_only · 20 unknown
Needs manual review	185	98 suitable_public_html · 65 unknown · 19 manual_only · 3 suitable_public_ats
No official source found	10	10 unknown
Blocked / unsafe	0	—

Categories A–G, computed from the live database

CATEGORY	COUNT	BASIS
A — Structured ATS, adapter exists today	17	verified + suitable_public_ats, on Greenhouse/Lever/Workable
A' — Structured ATS, no adapter yet	58	verified + suitable_public_ats, on Oracle/SAP/Workday/Taleo/SmartRecruiters/other
B — Careers page, appears scrapeable	358	suitable_public_html_subject_to_review — a researcher label, not a confirmed-scrapeable one
C — Careers page, mechanism unknown	95	automation_eligibility = unknown; the active Source Intelligence backlog
D — Redirects to another ATS	n/a	not separately tracked; folded into the destination's ats_provider
E — LinkedIn appears primary	0 in code	zero matches for "linkedin" anywhere in src/migrations/workflows; exists only as free-text researcher_notes on some CSV rows
F — No usable source found	10	no_official_source_found
G — Manual review only	185	needs_manual_review

THE NUMBER THAT MATTERS

"For how many companies do we have a technically usable ingestion method, today?" — **17 of 593 (2.9%)**. Extending to every verified ATS vendor with no adapter yet reaches **75 of 593 (12.6%)**. The other **~87%** is a genuine unknown that no amount of further registry research resolves by itself — it requires a decision among Sections 6–7.

ats_provider is free text, not an enum

59 distinct raw strings exist in `ats_provider` (e.g. "SmartRecruiters; Workday" as one compound value). `unknown` (402) and `none` (45) dominate. Any future adapter-routing logic keyed off this column needs normalization first, or it will silently miss compound/variant values.

04 Current ingestion architecture

One adapter per ATS type, not per company, not generic. A single manual-trigger n8n workflow (Job Ingestion — Pilot Orchestrator) hardcodes exactly four sources across three adapters (Greenhouse ×2, Lever ×1, Workable ×1).

MECHANISM	CURRENT BEHAVIOR
Source selection	Hardcoded static list of 4 <code>company_sources</code> ids; re-validates <code>review_status</code> / <code>automation_eligibility</code> live before every fetch — the static list is never itself trusted as authorization.
Discovery	None inside ingestion. Three separate, disconnected mechanisms feed the registry: one-shot CSV import (done), Registry Sync (manual/Sheets/Apify-dataset-read), Source Intelligence Analyzer (classifies existing unclassified rows). None writes <code>jobs</code> .
Normalization	Per-ATS field mapping inside the pilot only. <code>country_code</code> / <code>city</code> / <code>seniority</code> deliberately left NULL rather than guessed — no adapter's feed gives structured geography.
Deduplication	Generated column <code>dedup_scope</code> (source-specific, not ATS-type-specific) + unique index on (<code>dedup_scope</code> , <code>external_id</code>). Fixed a real cross-company collision bug. No story yet for the same job appearing via two different sources.
Canonical source	<code>jobs.source_id</code> → <code>company_sources.id</code> , nullable for admin-manual jobs. Solid.
Expired jobs	Soft-closed (<code>status='unavailable'</code>), never deleted, and structurally blocked from closing unless the fetch fully succeeded and wasn't truncated — prevents "one failed run nuked all jobs."
Retries	Real bounded exponential backoff with jitter, per adapter, <i>within a single run only</i> . No cross-run backoff state exists — a recurring scheduled worker would need to add this.
Rate limits	Fixed 2s placeholder delay between sources, not tuned per real provider limit. 429s are respected, not hammered.
Idempotency	Genuinely proven — re-running the pilot produces zero new/changed rows on unchanged input.
Scaling to hundreds/thousands	The static source list is a small, natural fix (query <code>company_sources</code> directly). The adapter ceiling is not: only 3 ATS families are coded, so scaling the source list alone still caps out at 75 sources (12.6%).

05 The main problem

This is not a scaling problem with the current strategy. It is a coverage ceiling.

An ATS-adapter strategy, however well generalized, tops out at whatever share of the registry actually sits behind a named ATS: **12.6%** at full extension. The other 87% — 358 "careers page, unconfirmed" sources, 95 still-unclassified sources, and 62 explicitly manual-only or unreachable sources — cannot be reached by writing more adapters, because there is no structured system on the other end to adapt to. Getting from 12.6% to something that makes the product viable requires choosing a fundamentally different mechanism (generic scraping, an AI/browser agent, a third-party provider, or accepting a much smaller addressable registry) for that 87%, or accepting that a meaningful fraction of the registry will simply never be automatable and should exit the registry rather than sit as dead weight.

06 Options considered

OPTION	REALISTIC COVERAGE	MAINTENANCE COST	VERDICT
A — ATS adapters	Caps at 12.6% of current registry; high reliability where it applies	Low per adapter, well-tested pattern already proven 3×	Keep extending — cheapest coverage available, do it first
B — Per-company scrapers	Could theoretically reach the 358 "careers page" sources	Very high — ~350 independent breakage points, no shared framework exists to amortize this	Do not build 1:1; unsustainable for a small team at this scale
C — Generic AI/browser agent	Could reach any HTML source in principle	Real hallucination and cost risk; needs deterministic validation before any write	Fallback / assisted-discovery only, never a trusted direct writer to <code>jobs</code>
D — Third-party provider (Apify etc.)	Broad, but provider- and pricing-dependent	Provider lock-in and ToS risk; already partially adopted for registry <i>discovery</i> , not job ingestion	Usable for discovery today; job-actor use needs its own evaluation
E — LinkedIn discovery	Potentially large, freshness unknown	Direct conflict with an existing product rule; compliance risk is real, not just technical	Founder decision required — see Section 10
F — Search-engine discovery	Gap-filler only, not a primary channel	Needs verification against an authoritative page before trust	Small, late-stage addition, not core
G — Hybrid, tiered	Sum of the above, applied where each is strongest	Higher orchestration complexity, but each tier stays simple in isolation	Recommended — see Section 7

07 Recommended architecture

A tiered router keyed off the source's own recorded capability — not a single ingestion method applied uniformly.

registry

Source Capability Registry

company_sources, extended with adapter_key / requires_browser / requires_provider / health columns — Section 11

↓

router

Source Router

reads the source's recorded capability, dispatches to exactly one tier below — no guessing at runtime

↓

tier 1

ATS adapters

Greenhouse/Lever/Workable today; extend to SmartRecruiters and other API-backed ATS as adapters are written

tier 2

Generic HTML adapters

templated per-domain extraction for the highest-value "careers page, unconfirmed" sources — built one shared parser at a time, not one bespoke scraper per company

tier 3

Third-party provider (Apify)

for sources where a maintained actor already exists and is cheaper than building/maintaining a scraper in-house

tier 4

AI/browser fallback

discovery-only — proposes structure and candidate listings; never writes to `jobs` without passing the same deterministic validation every other tier passes

tier 5

Manual / not automatable

sources that stay `manual_only` by design — no automation attempted

↓

shared

Normalize → Validate → Deduplicate → Canonicalize

one shared pipeline regardless of which tier produced the raw job — this already exists for Tier 1 and should not be rebuilt per tier

↓

db

jobs

with per-source health rolled back into the registry — last_success_at / failure_count — closing the loop

↓

unbuilt

Matching → Notifications

does not exist yet, independent of the ingestion decision — Section 13

08 Why this architecture

- **It matches what's already built.** Normalize/validate/dedup/canonicalize already exist and are tested for Tier 1. A router just needs to feed them from more sources — it does not replace them.
- **It isolates risk per tier.** A hallucinated AI extraction or a broken Apify actor can never reach `jobs` without passing the same validation a Greenhouse job passes. No tier is trusted more than the deterministic checks allow.
- **It's incremental and reversible.** Each tier can be built, measured, and possibly abandoned independently. Nothing requires committing to Apify or an AI agent before knowing whether Tier 1/2 alone gets close enough.
- **It stops treating "unknown" as a dead end.** Today a source with `automation_eligibility=unknown` has no next step defined. A router with defined tiers gives every source exactly one clear destination.

09 What we should not do

- Build ~350 one-off per-company scrapers. No shared framework exists to amortize the maintenance; this is the option every piece of evidence in this audit argues against.
- Let an AI/browser agent write directly to `jobs` on its own judgment. It may propose structure and candidates; only the existing deterministic validation may approve a write.
- Let today's shadow n8n workflows (Section 10) become the de facto architecture decision by default, just because they exist and work.
- Treat the CSV-era discovery-report.md as current — it is already flagged stale by the team's own later audit.
- Assume "verified" means "ingestible." 260 of the 398 verified sources are HTML-only with no confirmed extraction method — verification is a research/authorization signal, not a technical-capability one.
- Widen the static ingestion source list without also widening the adapter set — it only pushes the ceiling to 12.6%, not further.
- Declare the product's data problem solved once jobs start flowing. Matching, cover letters, and applications are still fully unbuilt.

10 LinkedIn / Apify decision

CLASSIFICATION — APIFY AS A DATASET-READ PROVIDER (ALREADY IN USE)

Use as one ingestion source, narrowly. Registry Sync's Apify mode only reads a dataset an operator already produced elsewhere — it cannot trigger or pay for a run. This is low-risk, already tested, and is being used for company/source *discovery*, not job ingestion. No change recommended here.

CLASSIFICATION — APIFY-TRIGGERED LINKEDIN JOB SCRAPING (THE NEW CAPABILITY BEING EXPLORED)

Research further — do not adopt by default. AGENTS.md §7 currently prohibits LinkedIn scraping outright. The evidence in this audit does not settle whether that rule should change; it only establishes that the technical shape described in the request (discover public listings, keep the original apply link, never automate the user's own LinkedIn account) is a materially different and lower-risk activity than what the existing rule was almost certainly written to prevent (scraping personal/authenticated LinkedIn data or auto-applying on a user's behalf). That distinction is real, but resolving it is a compliance and business decision, not a technical one — see Section 19.

WHAT WAS ACTUALLY FOUND — FLAG THIS DIRECTLY

Two workflows named "AI agent that can scrape webpages" and "Enrich LinkedIn profiles in Google Sheets with Apify" were created on the shared n8n instance **today**, untracked, inactive. This means someone has already begun prototyping in exactly this space, outside version control and ahead of this audit's recommendation. Per AGENTS.md, nothing here should be activated, published, or expanded without explicit sign-off — recommend reconciling what these actually do before any further work happens on them.

11 Source registry changes needed (conceptual — no migration)

`company_sources` is already a nascent capability registry, not a bare URL list — `ats_provider`, `automation_eligibility`, and `review_status` already exist and are actively used. Evolving it further is additive, not a rebuild.

CONCEPT	STATUS	GAP
source_url / source_type	exists	official_careers_url, ats_provider
ingestion_strategy	exists	automation_eligibility, though it's a research label today, not a router key
review_status	exists	—
adapter_key	missing	today the pilot hardcodes source→adapter mapping in the workflow itself, not the DB
requires_browser / requires_provider	missing	needed for the router in Section 7 to dispatch correctly
discovery_only	missing	needed to mark sources that feed discovery but never a direct job write
canonical_priority	missing	needed once the same job can arrive via two tiers — no dedup story exists cross-source today
last_success_at / last_failure_at / failure_count / last_job_seen_at / source_health	missing	<code>source_intelligence</code> tracks classification health, not ingestion health — no rollup exists today

This concept is architecturally appropriate. It should be an additive migration onto the existing table (`jobs.source_id` already points at `company_sources.id`, so nothing downstream needs to change), reviewed and reversible per AGENTS.md §17 — not built as part of this audit.

12 n8n workflow audit

WORKFLOW	CURRENT STATE	PROBLEMS	PRIORITY	RECOMMENDATION
Job Ingestion — Pilot Orchestrator	Manual trigger, inactive, 4 hardcoded sources, well-tested	No schedule; static list caps coverage regardless of registry growth	P0	Query-drive the source list, add a schedule, wire the Error Handler, activate
Source Intelligence Analyzer	Active live , uncommitted local edits	Zero high-confidence classifications produced to date — auto-promotion path is untested against reality; local/live drift risk right now	P0	Commit current work, verify live matches working tree before next scheduled run, investigate why confidence never reaches "high"
Registry Sync	Manual trigger, inactive, 3 adapters live-tested	No repo-visible record of the today's-date shadow workflows using the same Apify credential	P1	Reconcile against the two new live-only Apify/LinkedIn workflows before either is extended
Error Handler	Inactive, structurally sound, confirmed byte-identical repo→live	Notification boundary is a deliberate no-op; not yet attached as any workflow's Error Workflow	P1	Founder decision on wiring the existing <code>ai_job_agent_alerts_bot</code> Telegram credential, then attach to all scheduled workflows
CV Analysis Worker	Active live as "v2," repo has an older, differently-named, inactive version	Repo is not the source of truth for what's running; no visible AI spend cap	P0	Pull the live "v2" definition back into the repo before any further edits are made anywhere near CV analysis
AI/TECH NEWS AUTOMATION	Active live , zero repo representation	Unaudited, unversioned, unknown reliability/cost	P1	Import into repo as source-of-truth, or explicitly deprecate
"AI agent that can scrape webpages" / "Enrich LinkedIn profiles... with Apify" / "Generate leads with Google Maps" / "Lebanon Tech Hiring Intelligence"	Inactive, live-only, untracked, three created today	Directly overlap this audit's open questions; built ahead of any decision or review	P0	Reconcile with the team before any further work; do not activate pending Section 19

13 P0 blockers

- **Repo/live drift on CV Analysis Worker.** The committed workflow does not describe production behavior. Any edit made against the repo file right now risks being overwritten by, or conflicting with, the live "v2."
- **Undocumented, overlapping live workflows.** Three n8n workflows created today duplicate the exact scope of this audit, outside review. Left alone, they become the architecture by accident.
- **Zero matching/cover-letter/application implementation.** Independent of the ingestion decision — even a perfect ingestion pipeline produces no user value until this exists.
- **International Remote market has zero registry rows**, despite being a named target market.

14 P1 before production

- Source Intelligence Analyzer's auto-promotion path has never fired on real data — validate it against a larger sample before trusting it to unblock sources unattended.
- Cross-run backoff/rate-limit state for ingestion (currently only exists within a single execution).
- `jobs` schema gap: no `source_error` status distinct from `closed` / `expired` — a scheduled worker could wrongly close jobs on a transient failure.
- Error Handler's notification boundary wired to an actual channel, and attached to every scheduled workflow.
- Normalize the free-text `ats_provider` column before building any router logic that keys off it.

15 P2 — later

- Tier 2 generic HTML adapters for the highest-value "careers page, unconfirmed" sources.
- Cross-source dedup (same job via two sources) — not needed until Tier 2+ sources exist.
- Search-engine gap-filling discovery (Option F).
- Split the shared n8n instance if the co-located unrelated products become an operational or security concern.

16 Recommended next implementation step

THE ONE STEP

Convert the Pilot Orchestrator's hardcoded 4-source list into a live query against `company_sources` (`review_status='verified' AND automation_eligibility='suitable_public_ats' AND ats_provider IN (Greenhouse, Lever, Workable)` — the existing 17), attach the Error Handler as its Error Workflow, put it on a real schedule, and activate it.

This is the highest-leverage move because it uses **only code that is already built and already tested** — no new adapter, no new migration, no AI or third-party dependency. It turns the registry's real 2.9% into real, live, fresh jobs, which is the evidence needed to size every later decision (freshness decay rate, real failure rate, real per-source volume) instead of guessing. It also surfaces, immediately and cheaply, whether the completely-unbuilt matching layer is a bigger near-term blocker than ingestion breadth — which the evidence in this audit suggests it already is.

17 Implementation sequence

1. Activate the Pilot Orchestrator against a live registry query, scheduled, with the Error Handler attached (Section 16).
2. Reconcile CV Analysis Worker repo↔live drift and the three today-created shadow workflows before touching anything adjacent to them.
3. Add one more ATS adapter with a real public API (SmartRecruiters is the next-cheapest, per the live `ats_provider` counts) and re-measure coverage.
4. Evolve `company_sources` into the capability registry described in Section 11 — additive, reversible migration, reviewed.
5. Close the `jobs` schema gap (source_error status, last-checked distinction) flagged in the 2026-09-14 audit.
6. Build the minimal matching worker against real (even if small) job data — validates the full vertical slice end to end for the first time.
7. Only with real usage data in hand, evaluate Tier 2/3/4 investment (generic HTML adapters vs. Apify vs. AI/browser fallback) as a deliberate, costed decision.
8. Resolve the International Remote registry gap and the LinkedIn/Apify founder decision (Section 19) before any further work in that direction.

18 Business risk assessment

Can we realistically obtain enough fresh jobs for this business to work?

- **CONFIRMED** The current architecture, as coded today, reaches at most 12.6% of the known registry.
- **LIKELY** A hybrid tiered architecture (Section 7) can reach a meaningfully larger share — but no tier beyond ATS adapters has been built or measured, so the real number is not yet known.
- **UNCERTAIN** Whether Tier 2 (generic HTML) or Tier 3/4 (Apify/AI) close enough of the remaining 87% to matter, because none has been tried against this specific registry yet. This is answerable with a bounded pilot, not more analysis.
- **HIGH RISK** Shipping matching, cover letters, applications, and UI to a high standard while ingestion stays at today's 2.9–12.6% ceiling. Job supply is, on the evidence, **one of the two highest architectural risks in the product** — tied with the fact that nothing downstream of `jobs` has been built at all yet.

The scenario named in the request — strong CV analysis, matching, and UI but insufficient fresh jobs — is a real and currently unmitigated risk. It is not, however, a proven dead end: it is an unbuilt pipeline whose cheapest tier already works. The evidence supports treating this as solvable with deliberate, staged investment, not as a reason to abandon the product.

19 Questions requiring founder decision

COMPLIANCE

Should AGENTS.md's LinkedIn rule be narrowed to permit public-listing discovery (original apply link only, no login, no auto-apply), given that a materially different activity than the rule was likely written against is now technically on the table? The two shadow workflows found today make this decision time-sensitive.

SCOPE

Should Apify move beyond dataset-read (current, low-risk) into actor-triggered job scraping, and if so, who owns the ToS/compliance review and the ongoing spend?

PRIORITY

Ingestion breadth vs. building the entirely-missing matching/cover-letter/application pipeline — which gets engineering time first? This audit's evidence leans toward matching being the more urgent gap, but that's a product-priority call.

REGISTRY POLICY

Should the 72 sources that are `no_official_source_found` or `manual_only` get a public-application-email flow (permitted under AGENTS.md §7), or should they be retired from the registry rather than sit as unreachable?

DATA

Should the missing International Remote market data be imported now, given it's a named target market with zero current registry presence?

INFRASTRUCTURE

Should the n8n instance currently shared across three unrelated products be split, given the blast-radius and credential-hygiene questions this audit surfaced incidentally?

20 Final verdict

The engineering that exists is genuinely good — real idempotency, real retry design, a real dedup fix, real test coverage. It just covers a small slice of the problem, and the registry work, while valuable for research and eventual routing, was never going to answer the ingestion-mechanism question by itself.

Don't reach for AI agents or LinkedIn scraping first. Turn on what already works against the real registry, extend the cheap adapters, give the registry the capability fields it needs to route intelligently, and only then spend money and risk on the harder tiers — with the founder decisions in Section 19 made explicitly, not inherited by default from workflows someone already started building on the side.

Compiled from a live-database query session, direct reads of all n8n workflow source and test files, and a read-only comparison against the connected n8n instance. No files were modified, no workflow was executed or activated, no migration was run, and no secret values are reproduced anywhere in this report.